

2.

Pokazivači

U ovom ćemo se poglavlju upoznati s pokazivačima. Sigurno ste se nekad zapitali zašto prilikom upisivanja podataka s pomoću naredbe `scanf` uz varijablu u koju želimo spremiti neki sadržaj moramo pisati znak (operator) `&`. Ako ste ponekad zaboravili napisati taj operator, program nije radio. Sve je to bilo povezano s memorijom, tj. memorijskom lokacijom gdje su se spremale vrijednosti varijabli koje su bile korištene u programu.

```
scanf("%f %d", &a, &b);
```

Još smo u početku objasnili kako svaka varijabla kojom se koristimo u programu ima svoju vrijednost koja mora biti pospremljena na neku memorijsku lokaciju. Tijekom rada programa svi podatci i programski kod čuvaju se u RAM (engl. **Random Access Memory** – memorija s nasumičnim pristupom) memoriji. Jedna od karakteristika RAM-a je mogućnost da se svim memorijskim lokacijama pristupa u jednakom vremenskom intervalu. RAM nam omogućuje da podatke možemo upisivati u memoriju i čitati iz memorije.

Memoriju u računalu možemo predstaviti kao kontinuirani niz bajtova, a svaki bajt ima svoju adresu.

Tako na primjer memoriju koja zauzima 1 MiB možemo grafički prikazati kako je predočeno tablicom 2.1. Ako svaka lokacija zauzima 1 B to znači da možemo ukupno imati 2^{30} memorijskih lokacija. Memorijske lokacije (adrese) prikazane su dekadskim i heksadekadskim zapisom.

Tablica 2.1. Zauzeće memorije veličine 1 MiB

Adresa (ukupno ih ima 2^{30}) dek.vr. (hex.vr.)	Podatak (1 B)
0 (0x00000)	01011010
1 (0x00001)	10101011
2 (0x00002)	01011011
...	...
98771 (0x181D3)	10110101
98772 (0x181D4)	10111011
...	...
1048575 (0xFFFFF)	01011100

Koliko će mjesta neka varijabla zauzimati u memoriji, ovisi o tipu varijable (tablica 2.2). Naredbom `sizeof(tip_podatka)` možemo provjeriti koliko koji tip podatka zauzima mjesta u memoriji.

Tablica 2.2. Veličine osnovnih tipova podataka u C-u

Tip podatka	Veličina (bit)	Veličina (B)
char	najmanje 8	najmanje 1
short	najmanje 16	najmanje 2
int	najmanje 16	najmanje 4
long	najmanje 32	najmanje 4
float	32	4
double	64	8

2.1. Osnovni pojmovi o pokazivačima

Uvod o memoriji i što bolje razumijevanje memorijskog prostora gdje "žive" vrijednosti koje su spremljene u varijable puno će nam pomoći u daljnjim razmatranjima i radu s pokazivačima. Poznavanje rada s pokazivačima pomoći će vam kasnije kada ćete razvijati svoje programerske vještine i tehnike. Koristimo li pokazivače na pogrešan način, vrlo lako može doći do logičkih pogrešaka. Stoga s ovim "moćnim alatom" treba pažljivo.

2.1.1. Operator referenciranja

Prilikom deklaracije varijable, varijabli se pridružuje određen dio memorije čija veličina ovisi o tome kojeg je tipa varijabla. Pri samoj deklaraciji varijable, operacijski sustav tijekom izvođenja programa pridodjeljuje varijabli neku memorijsku lokaciju. U slučaju da želimo saznati na kojoj se adresi nalazi varijabla koju smo deklarirali, to radimo tako da ispred imena varijable napišemo znak *ampersand* (`&`). Znak `&` predstavlja adresni operator – **operator referenciranja**.

Primjerice, ako napišemo `y = &x`, to čitamo na sljedeći način: varijabli `y` pridruži adresu varijable `x`.

Ako u varijablu `x` koja je tipa `short` (zauzima 2 B) pohranimo cjelobrojnu vrijednost 2 i pretpostavimo da će taj podatak biti pohranjen na memorijskoj lokaciji koja ima adresu 92222, onda će to u memoriji izgledati kako je prikazano tablicom 2.5.

Tablica 2.5. Prikaz podataka koji je pohranjen u varijablu `x` u memoriji



Za dohvat vrijednosti adrese neke varijable koristi se **adresni operator `&`**.

2.1.2. Deklaracija i inicijalizacija pokazivača

Nakon što smo u prethodnom primjeru pristupili adresi varijable `x`, vrijednost te adrese je potrebno pohraniti u neku novu varijablu. Tu varijablu ćemo nazvati `px` i upravo će ta varijabla biti pokazivač (engl. *pointer*). Stoga kažemo da `px` pokazuje na varijablu `x` i sadržava adresu varijable `x`.

Na ovaj način smo napisali inicijalizaciju pokazivača. Sljedećom naredbom pridruživanja spremamo adresu varijable `x` u `px`:


```
px = &x;
```

pri čemu je:

px – varijabla u koju se sprema adresa varijable **x**; (**px** je pokazivač na **x**)

&x – adresa na koju je spremljena vrijednost varijable **x**.

Tablicom 2.6 prikazano je stanje u memoriji. Razlika u odnosu na tablicu 2.5 je ta da je u tablici 2.6 prikazana i varijabla (pokazivač) **px** koja isto mora biti spremljena negdje u memoriji. Vrijednost koju sadrži ta varijabla je adresa varijable **x**.

Pokazivač je varijabla koja sadrži adresu neke druge varijable u koju je pospremljen podatak.

Dosad smo pokazali kako možemo pomoću operatora referenciranja doći do vrijednosti adrese neke varijable. Da bismo mogli pomoću pokazivača izravno pristupiti vrijednosti na koju sam pokazivač pokazuje, prethodno je potrebno odrediti na koji tip podatka će pokazivač pokazivati.

To radimo deklaracijom pokazivača. Pokazivač se označava znakom zvjezdice ispred samog naziva pokazivača.

Deklaracija pokazivača:

```
tip *ime_pokazivaca;
```

Tip pokazivača treba odgovarati tipu varijable na koju pokazuje. Ako imamo varijablu tipa `short`, onda će pokazivač biti tipa `short *`. Deklaracijom pokazivača zauzet će se onoliki memorijski prostor koliko je definiran od strane prevoditelja koji se koristi. Mi ćemo u primjerima uzeti da se rezervira prostor veličine 4 B. Funkcijom `sizeof` je moguće provjeriti koliko je zauzeće u memoriji.

Sljedeći blok naredbi pokazuje kako smo varijabli `a`, koja je tipa `short`, pridružili vrijednost 2. Zatim smo deklarirali pokazivač `pa` koji je tipa `short *` te smo u trećoj naredbi inicijalizirali pokazivač (adresa varijable `a` pridružena je varijabli `pa`).

```
short a = 2;    //varijabli a tipa short pridruži vrijednost 2
short *pa;     //deklaracija pokazivača pa koji je tipa short *
pa = &a;       //inicijalizacija pokazivača (pa je pokazivač na a)
               - pokazivač pa sadržava adresu na kojoj se nalazi varijabla a
```

Tablica 2.6. U memoriji su prikazane varijable **x** i **px** na pretpostavljenim memorijskim lokacijama. Varijabla **x** sadržava podatak 2, a varijabla **px** (pokazivač na **x**) adresu varijable **x**. Brojevi 92222 i 2 su radi jednostavnosti zapisani dekadski u memoriji.

Adresa u memoriji	Podatak i vrijednost pokazivača u memoriji	
...	...	
92218	px	92222
...		
92221		
92222	x	2
92223		
...	...	

px je pokazivač na x

Prikazanu deklaraciju i inicijalizaciju pokazivača moguće je zapisati i na sljedeći način:

```
short a = 2;
short *pa = &a;    //pa deklariraj kao varijablu tipa pokazivač na short,
                  a potom deklariranu varijablu inicijaliziraj na vrijednost &a
```

U slučaju da ne znamo na koju adresu treba pokazivati pokazivač možemo ga inicijalizirati sa NULL npokazivačem na sljedeći način:

```
short *pa;    //deklaracija pokazivača pa koji je tipa short *
pa = NULL;    //null pointer pridruži pa
```

NULL pokazivač je simbolička konstanta. Definiran je unutar biblioteke `stdio.h`.

2.1.3. Operator dereferenciranja

Operatora dereferenciranja * omogućuje pokazivaču da pristupi vrijednosti varijable na koju pokazuje. Operator dereferenciranja nije isto što i * koju smo pisali prilikom deklaracije pokazivača.

Ako se vratimo na uvodni primjer u kojem je vrijednost varijable `x` bila 2, a adresa memorijske lokacije 92222 (tablica 2.6.) te napišemo sljedeće naredbe pridruživanja

```
int a = px;
int b = *px;
```

varijabli `a` bit će pridružena adresa varijable `x`: 92222, dok će varijabli `b` biti pridružena vrijednost 2 koja je spremljena na adresi 92222. Pomoću operatora dereferenciranja (`*px`) izravno smo pristupili vrijednosti varijable `x` na koju pokazuje pokazivač `px`.

Kako pristupiti vrijednosti varijable na koju pokazuje pokazivač proučit ćemo dodatno u primjeru 2.2.

Kada želimo ispisati vrijednost adrese neke varijable te vrijednost i adresu pokazivača, onda se koristimo sljedećim formatom za ispis: `%p`. Ispis će biti u obliku propisanom od strane prevoditelja (može biti dekadski, heksadekadski...). Sam oblik ispisa nije definiran standardom.

Operator dereferenciranja (`*`) i adresni operator (`&`) spadaju u unarne operatore i njihov je prioritet veći od aritmetičkih operatora.

Primjer 2.2.

Zadana je varijabla `x` u koju treba učitati neku cjelobrojnu vrijednost. Varijabla je tipa `int`. Napiši program koji će ispisati:

- vrijednost i adresu varijable `x`
- vrijednost i adresu pokazivača `px` na varijablu `x`.

Napiši sve potrebne deklaracije varijable `x` i pokazivača `px` te inicijaliziraj pokazivač `px` tako da pokazuje na `x`.

```
#include <stdio.h>
int main(void){
```

```
    int x;           //deklaracija varijable x
    int *px;         //deklaracija pokazivaca px
    px = &x;         //inicijalizacija pokazivača
```

```
    printf("Učitaj cjelobrojnu vrijednost u varijablu x: ");
    scanf("%d", &x); //učitavanje vrijednosti u varijablu x
    printf("\nVrijednost varijable x je %d", *px);
    printf("\nAdresa varijable x je %p", &x);
```

```
    printf("\nVrijednost pokazivaca px je %p", px);
    printf("\nAdresa pokazivaca px je %p", &px);
```

```
    return 0;
```

```
}
```

ova zvjezdica ne predstavlja operator dereferenciranja, već označava da je varijabla `px` pokazivač na `int`

ova zvjezdica predstavlja operator dereferenciranja

Ispis na zaslonu računala izgleda ovako:

```
Učitaj cjelobrojnu vrijednost u varijablu x:2
Vrijednost varijable x je 2
Adresa varijable x je 0022FF44
Vrijednost pokazivaca px je 0022FF44
Adresa pokazivaca px je 0022FF40
```

Na temelju ispisanih adresa prikaz u memoriji računala izgleda kako je prikazano tablicom 2.7.

Tablica 2.7. Prikaz u memoriji računala za primjer 3.2

Adresa u memoriji	Podatak i vrijednost pokazivača u memoriji	
...	...	
0022FF40	px	0022FF44
0022FF44	x	5
...	...	